

DevOps y pruebas automáticas

DevOps

¿Qué es DevOps?

DevOps es una cultura que promueve la colaboración entre el equipo de desarrollo y el de operaciones para implementar el código en producción más rápido de forma automatizada y repetible. Además, facilita la comunicación, colaboración e integración entre el equipo de desarrolladores y el equipo de operaciones. La palabra 'DevOps' es una combinación de dos palabras 'desarrollo' y 'operaciones'.



¿Por qué se necesita DevOps?

- Antes de DevOps, el equipo de desarrollo y operación trabajaba en completo aislamiento.
- Sin usar DevOps, los miembros del equipo dedican una gran cantidad de su tiempo a probar, implementar y diseñar en lugar de construir el proyecto.
- La implementación manual de código conduce a errores humanos en la producción.



¿Por qué se utiliza DevOps?

DevOps permite que los equipos de desarrollo ágiles implementen la integración continua y la entrega continua. Esto les ayuda a lanzar productos más rápidamente al mercado.

1) Previsibilidad

DevOps ofrece una tasa de fallas significativamente menor de los nuevos lanzamientos.

2) Reproducibilidad

Versione todo para que la versión anterior se pueda restaurar en cualquier momento.

3) Capacidad de mantenimiento

Proceso de recuperación sin esfuerzo en caso de que una nueva versión falle o desactive el sistema actual.

4) Tiempo de comercialización

DevOps reduce el tiempo de comercialización hasta en un 50% mediante la entrega de software optimizada. Este es particularmente el caso de las aplicaciones digitales y móviles.

5) Mayor calidad

DevOps ayuda al equipo a mejorar la calidad del desarrollo de aplicaciones, ya que incorpora problemas de infraestructura.

6) Riesgo reducido

DevOps incorpora aspectos de seguridad en el ciclo de vida de la entrega de software. Ayuda a reducir los defectos a lo largo del ciclo de vida.

7) Resistencia

El estado operativo del sistema de software es más estable, seguro y los cambios son auditables.

8) Rentabilidad

DevOps ofrece rentabilidad en el proceso de desarrollo de software, que siempre es una aspiración de la gestión de las empresas de TI.

9) Divide la base de código más grande en partes pequeñas

DevOps se basa en el método de programación ágil. Por lo tanto, permite dividir bases de código más grandes en fragmentos más pequeños y manejables.



¿Cuándo adoptar DevOps?

DevOps debe usarse para grandes aplicaciones distribuidas, como sitios de comercio electrónico o aplicaciones alojadas en una plataforma en la nube.

¿Cuándo no adoptar DevOps?

No debe usarse en una aplicación de misión crítica como bancos, energía y otros sitios de datos confidenciales. Dichas aplicaciones necesitan controles de acceso estrictos en el entorno de producción, una política de gestión de cambios detallada, una política de control de acceso a los centros de datos.

Ciclo de vida de DevOps

DevOps es una integración profunda entre el desarrollo y las operaciones. No es posible comprender DevOps sin conocer el ciclo de vida de DevOps.

Aquí hay una breve información sobre el ciclo de vida de DevOps continuo:

1) Desarrollo

El desarrollo de software se lleva a cabo constantemente.

2) Prueba

Se utiliza automatización de *testing*.

3) Integración

La nueva funcionalidad se integra con el código vigente y se llevan a cabo las pruebas. El desarrollo continuo solo es posible gracias a la integración y las pruebas continuas.

4) Despliegue

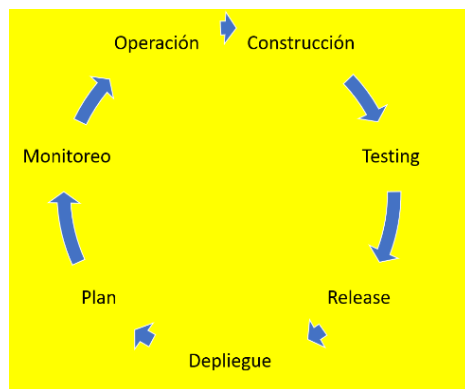
Proceso de implementación se lleva a cabo de forma continua.

5) Seguimiento

Proceso control del comportamiento inadecuado del sistema.

6) Flujo de trabajo de DevOps

Proporciona el flujo de trabajo.



Entrega continua e integración continua

La entrega continua se centra en hacer que el desarrollo de un producto esté siempre en un estado de entrega a lo largo de su ciclo de vida. La entrega continua mejora la eficiencia y ajusta la planificación y el presupuesto del proceso de entrega de software, por lo que es más barato y conlleva menor riesgo liberar nuevas versiones del software al cliente.

La integración continua es una práctica de desarrollo por el que los desarrolladores fusionan de forma rutinaria su código en la rama central en un sistema de control de versiones - idealmente, varias veces por día. Cada cambio desencadena un conjunto de pruebas rápidas para descubrir posibles errores, que los desarrolladores deben solucionar de inmediato. Este proceso es en realidad el primer paso para lograr la entrega continua.

El objetivo de la integración y entrega continua es hacer que el proceso de liberación de los cambios al cliente final sea técnicamente sencillo, es decir, un proceso rutinario y aburrido. Llegados a este punto, el equipo de TI puede dedicar más tiempo a tareas de planificación y estrategias proactivas que pueden producir aún más valor a la empresa.

Principios de DevOps

Aquí hay seis principios que son esenciales al adoptar DevOps:

1) Acción centrada en el cliente

El equipo de DevOps debe realizar una acción centrada en el cliente para que inviertan constantemente en productos y servicios.

2) Responsabilidad de un extremo a otro

El equipo de DevOps debe brindar apoyo al desempeño hasta que llegue al final de su vida útil. Esto mejora el nivel de responsabilidad y la calidad de los productos diseñados.

3) Mejora continua

La cultura DevOps se centra en la mejora continua para minimizar el desperdicio. Acelera continuamente la mejora de los productos o servicios ofrecidos.

4) Automatizar todo

La automatización es un principio vital del proceso DevOps. Esto no es solo para el desarrollo de software, sino también para todo el panorama de la infraestructura.

5) Trabajar como un solo equipo

En la cultura DevOps, el rol del diseñador, desarrollador y evaluador ya está definido. Todo lo que necesitaban hacer es trabajar como un solo equipo con total colaboración.

6) Monitorear y probar todo

Es muy importante para el equipo de DevOps tener procedimientos sólidos de monitoreo y prueba.

¿Cuál es el futuro de DevOps?

Es probable que ocurran muchos cambios en el mundo de DevOps, algunos de los más destacados son:

- Las organizaciones están cambiando sus necesidades a semanas y meses en lugar de años.
- Pronto veremos que los ingenieros de DevOps tienen más acceso y control del usuario final que cualquier otra persona en la empresa.
- DevOps se está convirtiendo en una habilidad valiosa para el personal de TI. Por ejemplo, una encuesta realizada por contratación de Linux encontró que el 25% de los solicitantes de empleo de los encuestados tiene experiencia en DevOps.
- DevOps y la entrega continua llegaron para quedarse. Por lo tanto, las empresas deben cambiar, ya que no tienen más remedio que evolucionar. Sin embargo, la integración de la noción de DevOps llevará de 5 a 10 años.

Resumen

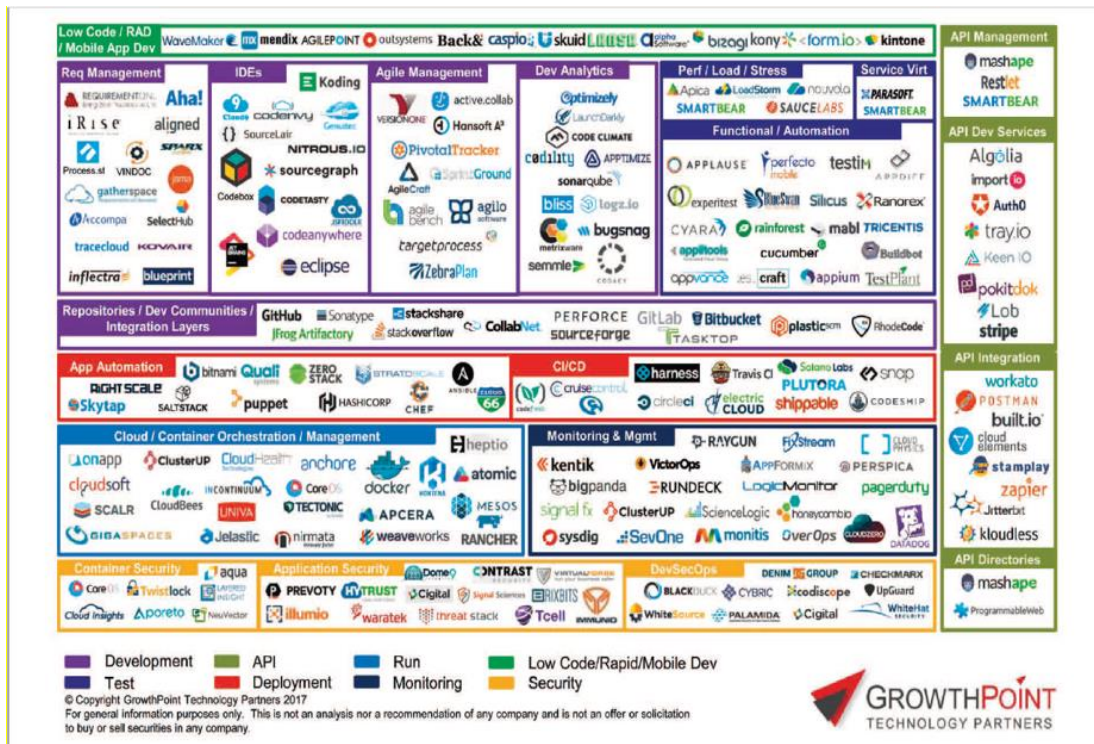
- DevOps es una cultura que promueve la colaboración entre el equipo de desarrollo y el de operaciones para implementar el código en la producción más rápido de forma automatizada y repetible.
- Antes de que el equipo de desarrollo y operaciones de DevOps trabajara en completo aislamiento.
- La implementación manual de código conduce a errores humanos en la producción.
- En el proceso anterior, el equipo de operaciones no tiene ni idea del progreso del equipo de desarrollo. Entonces, el equipo de operaciones desarrolló un plan de monitoreo y compra de infraestructura de TI según su entendimiento.
- En el proceso de DevOps, el equipo de operación es plenamente consciente de los avances del desarrollador. La planificación de compras y seguimiento es precisa.
- DevOps ofrece capacidad de mantenimiento, previsibilidad, mayor calidad, rentabilidad y tiempo de comercialización.

- ## Herramientas actuales de DevOps

PERIODIC TABLE OF DEVOPS TOOLS (V2) EMBED DOWNLOAD ADO

Legend:

- SCM: Gh, Gt, Dt, Bb, Lb
- Database Mgmt: Gl, Rg, Mv, Gr, At, Fn, Se, Ga, Dh, Jn, Ba, Tr, Gd, Sf, Cn, Bc, Mo, Rs
- Build: Sv, Dm, Gn, Gp, Br, Cu, Cj, Qu, Npm, Cs, Vs, Cr, Cp, Ju, Rd, Cf, Ds, Op
- CI: Hg, Dp, Sb, Mk, Ck, Jt, Jm, Tn, Ay, Tc, Sh, Cc, Ry, Cy, Oc, No, Kb, Hr
- Deployment: Cw, Id, Msb, Rk, Pk, Mc, Km, Jm, Nx, Co, Ct, So, Xld, EB, Dp, Ud, Nm, Oo, Ff
- Config/Provisioning: Xl, Release, Ur, UserCode, Bm, BmC, Ca, CaRelease, Au, AuAutomatic, Pl, PlFuture, Sr, SrSereni, Tf, TfTeam, Ti, TiTrello, Jr, JrJira, Rf, RfHip, Si, SiStack, Fd, FdFlask, Pv, PvPulumi, Sn, SnService
- Containerization: Ki, KiK8s, Nr, NrNew, Dt, DtDocker, Ni, NiNagios, Zb, ZbZabbix, Dd, DdDatadog, Ei, EiElasticsearch, Ad, AdApp, Sp, SpSplunk, Le, LeLogentries, Si, SiSumo, Ls, LsLogstash, Sn, SnSnort, Tw, TwTwilio, Ff, FfFifty
- Release Mgmt: Xl, Release, Ur, UserCode, Bm, BmC, Ca, CaRelease, Au, AuAutomatic, Pl, PlFuture, Sr, SrSereni, Tf, TfTeam, Ti, TiTrello, Jr, JrJira, Rf, RfHip, Si, SiStack, Fd, FdFlask, Pv, PvPulumi, Sn, SnService
- Collaboration: Ki, KiK8s, Nr, NrNew, Dt, DtDocker, Ni, NiNagios, Zb, ZbZabbix, Dd, DdDatadog, Ei, EiElasticsearch, Ad, AdApp, Sp, SpSplunk, Le, LeLogentries, Si, SiSumo, Ls, LsLogstash, Sn, SnSnort, Tw, TwTwilio, Ff, FfFifty
- Security: Ki, KiK8s, Nr, NrNew, Dt, DtDocker, Ni, NiNagios, Zb, ZbZabbix, Dd, DdDatadog, Ei, EiElasticsearch, Ad, AdApp, Sp, SpSplunk, Le, LeLogentries, Si, SiSumo, Ls, LsLogstash, Sn, SnSnort, Tw, TwTwilio, Ff, FfFifty



¿Qué es DevSecOps?

DevSecOps es integrar la capa de seguridad desde el principio dentro de la metodología DevOps. Establece que tanto los desarrolladores deben ocuparse de la seguridad desde el principio para:

- Evitar fallos y errores.
- Mejorar la rentabilidad
- Aumentar la fidelidad y la satisfacción de los clientes.

La metodología DevSecOps, como DevOps, se apoya considerablemente en la automatización. De modo que, todos los procesos de seguridad que se puedan automatizar, como las auditorías de seguridad, se deberían automatizar.



Ciclo de vida DevSecOps

Buenas prácticas de DevSecOps:

- Mejorar los procesos de seguridad y desarrollo.
- Enseñar buenas prácticas a los empleados; ya que los usuarios son el punto más débil de cualquier sistema IT.
- Gestionar y mejorar el control de acceso.
- Automatizar procesos repetitivos.
- Evaluar riesgos y definir un plan de contingencia y DR.
- Utilizar herramientas de seguridad IT.
- Buscar amenazas y vulnerabilidades de forma proactiva.
- Realizar auditorías de seguridad periódicamente.

La importancia de DevSecOps en el *cloud*

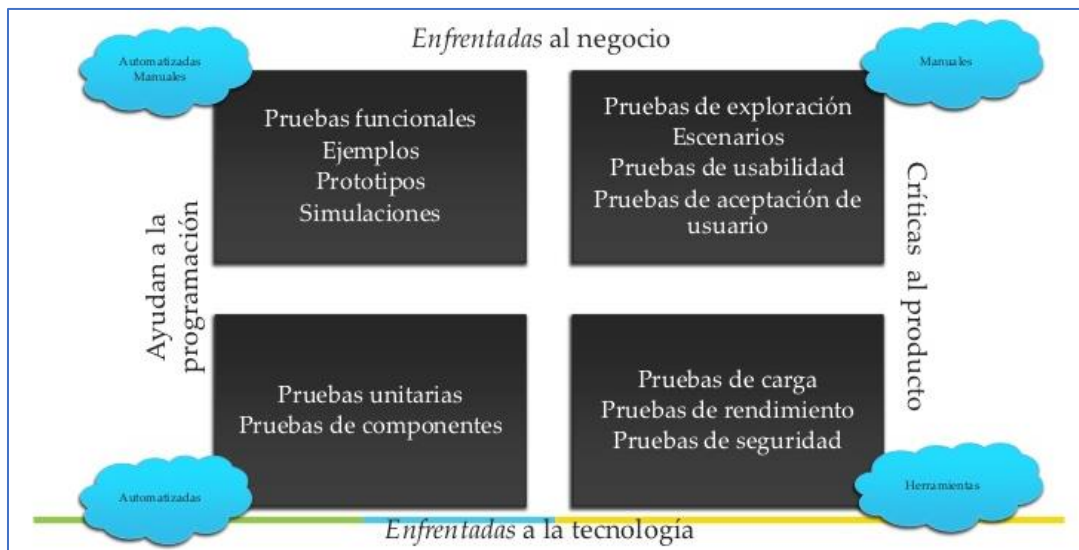
Al planear una migración a la nube, la seguridad debe abordarse cuidadosamente para evitar riesgos y ahorrar tiempo y dinero. Al externalizar la infraestructura IT, las empresas suelen confiar sus datos más sensibles a un proveedor externo. Así que la seguridad se debería evaluar y abordar en cada etapa. Los servicios *cloud* están basados en un modelo de responsabilidad compartida, por lo que tanto el proveedor de servicios como el cliente son responsables de garantizar el mejor nivel de seguridad posible.

Pruebas automáticas

1) Pruebas automáticas

Un método de ejecutar una prueba sin intervención humano, que de lo contrario lo requeriría.

2) Cuadrante de agile testing



Cuadrante 1

Encontramos las pruebas unitarias y de componente. Estas pruebas se deberían intentar automatizar en el mayor porcentaje posible, para conseguir una situación de control sobre el trabajo ya desarrollado que aporte confianza al equipo para continuar trabajando "hacia delante".

Cuadrante 2

En este cuadrante encontramos las pruebas que, aun no siendo unitarias, prueban el sistema a un nivel más técnico que las pruebas que pudiera realizar un usuario final.

Cuadrante 3

Aquí se reúnen las pruebas que se han de realizar de forma manual, y en las que enfocándonos en el negocio, probaremos la aplicación con un espíritu crítico. A diferencia de los anteriores cuadrantes, las pruebas de este tendrán el objetivo de descubrir

información sobre la aplicación y su comportamiento y no validar únicamente que se cubre las expectativas del programador.

Cuadrante 4

En este cuadrante se engloban las pruebas de rendimiento y todas las pruebas que puedan ser necesarias en el proyecto (usabilidad, seguridad, estabilidad, etc.).

3) ¿Por qué automatizar?

La principal razón es el tiempo y con las pruebas automatizadas se puede reducir el tiempo de las pruebas. Además, al automatizar las actividades comunes que no requieren de inteligencia humana, los *testers* reales pueden dedicar mayor tiempo a pruebas más críticas y caminos más elaborados dejando los caminos básicos a las pruebas automatizadas.

4) Atributos de automatización de pruebas

Conciso

La prueba debe ser lo más sencillo posible y simple.

Comprobación computacional

La prueba debe reportar sus resultados de forma que no se necesite interpretación humana.

Repetible

La prueba se puede ejecutar varias veces sin la intervención humana.

Robusto

Test produce el mismo resultado ahora y para siempre. Las pruebas no se ven afectadas por cambios en el entorno externo.

Necesario

Todo en cada prueba contribuye a la especificación deseada de comportamiento.

Despejado

Cada afirmación es fácil de entender.

Eficiente

Las pruebas realizadas en un período razonable de tiempo.

Específico

Cada punto de falla de prueba a una parte específica de la funcionalidad rota (por ejemplo, cada caso de prueba comprueba un posible punto de fallo).

Independiente

Cada prueba se puede ejecutar por sí mismo o en una suite con un conjunto arbitrario de otras pruebas en cualquier orden.

Sustentable

Las pruebas deben ser fáciles de modificar y ampliar.

Trazable

Las pruebas deben ser conforme a los requisitos, los requisitos deben ser trazable a las pruebas.

5) *¿Qué casos de prueba se deberían automatizar?*

- Casos de prueba que se deban correr en cada nueva versión de la aplicación (*Sanity Testing*).
- Casos de prueba que utilicen distintos datos de prueba para las mismas acciones. (*Data Driven Testing*).
- Funcionalidades críticas de la aplicación (*Smoke Testing*).
- Funcionalidades que no cambiaran en un periodo de tiempo relativamente corto.
- Escenarios de pruebas de performance.

6) *¿Cuáles son las desventajas de la automatización?*

- Las herramientas de *testing* automatizado no pueden medir la usabilidad de una aplicación.

- Se requieren conocimientos en programación para poder adaptar los scripts automatizados a los requerimientos.
- El mantenimiento de los scripts puede ser muy costoso.

7) ¿Qué cantidad de ciclos de prueba debe haber?

- Las pruebas automatizadas tienen más valor cada vez que se corren, por este motivo aquellos proyectos que involucren varios ciclos de prueba son los que sacan más valor de este tipo de pruebas.

8) ¿Qué tecnología se debe utilizar?

Dependiendo de la tecnología que se utilice para desarrollar existen herramientas para automatizar de menor o mayor complejidad y de menor o mayor precio.

Las herramientas de *testing* automatizado pueden servir no solo para automatizar casos de prueba, sino que también para generar datos para los casos de prueba. Por este motivo conocer de la herramienta de automatización ayudará a también en las pruebas manuales.

9) ¿Cómo formar a las personas que van a automatizar?

El proceso que siga cada una de las personas que vaya a automatizar puede variar según su grado de conocimiento en *testing* automatizado y en herramientas de automatización. Es aconsejable que de manera paralela puedan comenzar con la capacitación en la herramienta que se vaya a utilizar, así como también leyendo material acerca de metodología y experiencias de pruebas automatizadas en general.

10) ¿Cómo seleccionar los casos de prueba a automatizar?

El error más común es intentar automatizar todo, **no hay que intentar automatizar todos los casos de prueba.**

Teniendo una visión del negocio y también de la estructura interna del sistema (para lo cual se necesita involucrar a los desarrolladores) se debe decidir cuales casos de prueba que se automatizarán.

Algunas de los factores a tener en cuenta son:

¿Cuáles casos de prueba se necesitarán correr más veces durante el proyecto?

Típicamente aquellos casos de prueba asociados al núcleo de la aplicación se pueden construir al principio del proyecto y luego sirven (adaptándolos cuando sea necesario) para el resto de la vida del sistema.

¿Cuáles casos de prueba conllevan mucha dedicación humana y son repetitivos?

Hay veces que un caso de prueba requiere de un trabajo tedioso para configurar el ambiente de prueba y luego ejecutar el caso de prueba. Estos casos de prueba es bueno automatizarlos para liberar el recurso humano y que pueda dedicarse a otros casos más desafiantes.

¿Qué funcionalidades son críticas para el cliente/usuario?

Aquellas funcionalidades que tienen que andar si o si en cada entrega generalmente es interesante incluirlas en los casos automatizados.

¿Qué costo tienen asociado la automatización del caso de prueba?

Hay casos de prueba que son muy difíciles de automatizar, ya sea porque hay que realizar validaciones visuales complejas o por otras razones. Por este motivo si el costo de su automatización es grande hay veces que es mejor dejarlo para ser ejecutado manualmente.

Teniendo en cuenta estas características se puede ponderar cada caso de prueba en base a cada una de ellas para luego hacer un ranking o priorizar los casos de prueba a automatizar.

11) Tabla de comparación de Testing Automatizado vs Testing Manual

Testing Automatizado	Testing Manual
Ventajas	Ventajas
1) Si se tienen que correr un juego de pruebas repetidamente, la automatización será de muchísima	1) Si el caso de prueba sólo se ejecuta dos veces ante un cambio en la codificación, probablemente debería ser una prueba

ayuda.	manual antes que automatizada ya que el costo es mayor al beneficio.
2) Nos permite correr las pruebas automatizadas sobre un código que cambia frecuentemente reduciendo el tiempo insumido en las pruebas de regresión.	2) Permite al tester ampliar más las pruebas durante la ejecución del caso de prueba. Encontrando más bugs ya que el tester puede inventar combinaciones impensadas mientras navega la aplicación y ante diversas situaciones.
3) Permite realizar matrices de pruebas combinando diferentes lenguajes con diferentes SO.	
4) Permite realizar pruebas en paralelo en una o varias máquinas.	
Desventajas	Desventajas
1) El esfuerzo inicial es mayor. La creación del script automatizado puede ser más costoso que la creación del caso de prueba para ejecución manual.	1) Las pruebas manuales pueden consumir demasiado tiempo.
2) No se pueden (o puede ser muy costoso) automatizar referencias visuales como colores o ubicación en pantalla de objetos.	2) Cada vez que hay un cambio en la aplicación el tester debe correr las pruebas de regresión. Ante reiterados cambios, el tester puede llegar a correr demasiadas veces las pruebas de regresión, reduciendo su capacidad de encontrar errores.

12) Tipos de herramientas de testing

La selección de herramientas es un trabajo que cualquier líder de un proyecto de *testing* deberá utilizar. Por ello se debe familiarizar las herramientas del mercado y trabajar en como asociar sus características con las actividades estándar de *testing*.

A continuación, describo varias herramientas del mercado con algunas características de ella agrupadas en las utilizadas en la gestión del *testing*, *UI Testing* y automatización.

Gestión del *testing*

Systir

- Testeo de sistemas especificando un lenguaje orientado al dominio.
- El tester utiliza este lenguaje (creado con Ruby).
- Esta especificación es ejecutable.
- Separación entre la especificación del test y la implementación.
- *Browse*.

TestLog

- Básica y fácil de usar.
- Repositorio XML (no BD SQL).
- *Free*.

Salomé – TMF

- Basado en el estándar para *testing* ISO9646:
 - *Test Family*
 - *Test Suite*
 - *Test*
 - *Dataset*
 - *Execution*
 - *Test Campaign*
 - *Environment*
- El test (case) puede ser manual o automático (JUnit, Abbot, CLIF, otro).
- El test automático es un script que será ejecutado por la herramienta y el resultado registrado en la campaña de prueba correspondiente.
- *Free*.

Test Track (Seapine)

- *Workbook*: dashboard de tareas, test cases, defects y runs asignados. Se pueden agregar *tabs*.
- *Folders*: vista que permite organizar la información usando *drag&drop*.
- *Custom fields*.
- *Linking defects*.
- *Automation rules*.
- RSS.
- Soporta automatización mediante *Automation Rules* y *workflow*.

SQADesign

- Método *Bottom-Up*.
- Buena captura de UI (opcional) – no macros.
- Casos de Uso (*scenarios*).
- *Test Step* (opcionales - UI o no).

Conexión con JIRA

- A nivel proyecto.
- Asociación de *Test Cases*, *Scenarios*, y *Test Plans* con *JIRA issues*.
- Carga de cualquier tipo de *Issue*.
- Búsqueda de *issues*.
- Hecha en .Net y SQL server.
- Fácil de usar.

Xstudio

- Basada en modelo ISO.
- Interface basada en árboles.
- Administra requerimientos.
- Permite distribuir la ejecución de test suites en varias PC.
- Localizado para varios idiomas.
- No hay conexión con JIRA.
- Soporta *parameters* (y *checks*).
- Trazabilidad: un test se asocia a una *specification* (y esta a los requerimientos).
- Tiene una opción para generar **requirement book** (configurables vía XSLT).

- Importación: CSV, XML. Exportación: XML.
- Dependencias entre *tests* (*execute only child tests if the parents are all successful*).
- Lástima la implementación (deja ventana MS-DOS abierta).
- *Free*.

TestLink

- Última versión instalada en producción.
- Integración con JIRA (*defects*).
- ¿Automatización? (sólo permite leer los resultados de *script* externo).

UI testing

Squish

- Testing Java GUI applications based on SWT, RCP/Eclipse, Swing and AWT.
- Los test pueden ser manuales o capturados (*recording*).

Borland The Open Alm Company

- Cada test case es independiente del resto.
- Ambiente de datos para cada uno.
- Se registra los test cases a partir de la UI y los valores esperados.

Abbot es un *framework* de *testing* programático tipo JUnit. De esta manera, se puede realizar *testing* de regresión del look de la UI.

Salomé-TMF, que también administra casos de prueba JUnit y manuales, usa Abbot.

Automatización

Fitnessse

- Framework de test integrado a un wiki.
- Prueba de aceptación en lugar de unidad.
- Usa FIT:
 - Especificación casos de prueba en MS-Excel.
 - Implementación en JUNIT.
- Filosofía de y para métodos ágiles.

TestMentor

- Administración, y generación de la automatización de componentes Java.
- Orientado a ser un puente entre los *testers* y los *developers*, permite definir *assets* de *testing* con las herramientas.
- Los test que se crean manualmente o que se generan pueden representarse en forma de código Java o en forma visual como activos de test.
- Muy completa.
- Lástima que no se integra con Eclipse IDE.
- Integra JUnit.

iValidator

- Framework de testing para Java.
- Se codifica las unidades de test en Java.
- Describir escenarios de prueba.
- Ejemplo:
 - Adapter to connect to the SUT
 - *Units to perform tests*
 - XML-TestDescriptions for several scenarios
 - *Test Runner*
- Se integra con JUnit.
- Existe *plugin* para Eclipse.

JML / ModernJass

- *Java Modeling Language*: lenguaje alto nivel diseñar por contratos con Java.
- Se usa tanto para diseñar como para *testing* (se puede reemplazar ciertos Junits).
- Pre/Post *Conditions*. Invariantes.
- Compilador, Analizador estático, *Runtime Checker*.
- *Plugins* para Eclipse.
- Modern Jass: una implementación de JML5 que usa Java 5 *annotations*.
- Interesante para definición de contratos de subsistemas, programación defensiva sin afectar performance, mejor aprovechamiento de JUnits, testing funcional, componentes, exploratorio (*runtime*).